

EC4219: Software Engineering

Lecture 12 — Type System (1) *Introduction*

Sunbeom So
2025 Spring

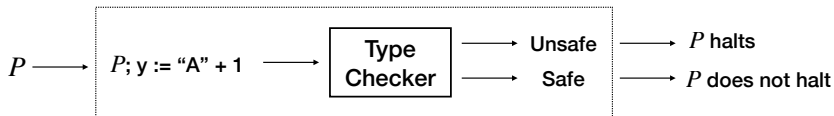
- **Type:** the set of values that each expression can take on. E.g.,

int = $\{\dots, -2, -1, 0, 1, 2, \dots\}$, **bool** = $\{true, false\}$

- **Type system:** a set of rules for defining types of expressions and validating whether evaluations of the expressions are **type-safe** – that is, whether types of expressions match the expected ones.
 - ▶ Type formation rules: a set of rules for constructing types. E.g:
 $list \rightarrow \mathbf{nil} \mid n \cdot list \ (n \in \mathbb{Z})$
 - ▶ Typing rules: a set of rules for assigning types. E.g.: “if E_1 and E_2 have type int, then $E_1 + E_2$ also has type int”.
 - ▶ Type checking: algorithms for ensuring type safety.
 - ▶ Automatic type inference: algorithms for automatically deducing the types of expressions without requiring explicit type annotations.
 - ▶ ...

Type Analysis is Undecidable

- (Claim) We cannot have an ideal type checker that can always produce a correct answer (type-safe or type-unsafe) for any program.
- (Proof by Contradiction) Suppose exact type checking is possible. Then, we can solve the Halting problem using it!



- More precisely, **sound** and **complete** type analysis is impossible.
 - ▶ **Sound**: can prove the absence of type errors. Every type-unsafe program is not accepted. Can detect all type errors.
 - ▶ **Complete**: can accept every type-safe program.
- Modern type checkers compromise soundness or completeness.

We will explore a sound but incomplete, and static type system.

- incomplete: some safe programs will not pass our type system.
 - ▶ Type systems in modern functional programming languages such as OCaml, Haskell, and Scala are also sound but incomplete.
 - ▶ Type systems in languages like C and C++ are neither sound nor complete.
- static: type checking is done at compile-time. Examples: C, C++, Java, ML, Scala.
 - ▶ In dynamically typed languages, type checking is done at run-time. Examples: Python, JavaScript, Ruby, Lisp, etc

Syntax of Language

$$\begin{array}{lcl} E & \rightarrow & n \ (\in \mathbb{Z}) \\ & | & x \\ & | & E_1 + E_2 \\ & | & E_1 - E_2 \\ & | & \text{iszero } E \\ & | & \text{if } E_1 \text{ then } E_2 \text{ else } E_3 \\ & | & \text{let } x = E_1 \text{ in } E_2 \\ & | & \text{proc } x \ E \\ & | & \text{letrec } f(x) = E_1 \text{ in } E_2 \\ & | & E_1 \ E_2 \end{array}$$

- $E_1 \ E_2$: function application. Read as “ E_1 (is) applied to E_2 ”.
- We assume the usual semantics. For example:
 - ▶ If E_1 and E_2 evaluate to 3 and 5, $E_1 + E_2$ evaluates to 8.
 - ▶ $(\text{proc } x \ x) \ 1$ evaluates to 1.

Types

Types are defined inductively:

$$\begin{array}{lcl} T & \rightarrow & \text{int} \\ & | & \text{bool} \\ & | & T \rightarrow T \end{array}$$

The following types can be constructed:

- int
- bool
- $\text{int} \rightarrow \text{int}$
- $\text{bool} \rightarrow \text{int}$
- $\text{int} \rightarrow (\text{int} \rightarrow \text{bool})$
- $(\text{int} \rightarrow \text{int}) \rightarrow (\text{bool} \rightarrow \text{bool})$
- $(\text{int} \rightarrow \text{int}) \rightarrow (\text{bool} \rightarrow (\text{bool} \rightarrow \text{int}))$

Type Environment

- To compute the type of an expression, we introduce **type environment** Γ :

$$\Gamma : Var \rightarrow T$$

- Type environment is an abstraction of (infinitely many) states at runtime. For example, $\{x \mapsto \text{int}, y \mapsto \text{int}\}$ abstracts $\{x \mapsto 0, y \mapsto 0\}, \{x \mapsto 1, y \mapsto 0\}, \dots, \{x \mapsto 999, y \mapsto 999\}$.

Notations:

- $[]$ (or $\emptyset$): the empty type environment
- $\{x \mapsto t\}\Gamma$: the extended type environment where x has type t .
- $\Gamma(x)$: the type of x under Γ .
- $\Gamma \vdash e : t \Leftrightarrow$ "Under type environment Γ , expression e has type t ."

Typing Rules

Inductive proof rules for assigning types to expressions:

$$\frac{}{\Gamma \vdash n : \text{int}} \quad \frac{}{\Gamma \vdash x : \Gamma(x)}$$

$$\frac{\Gamma \vdash E_1 : \text{int} \quad \Gamma \vdash E_2 : \text{int}}{\Gamma \vdash E_1 + E_2 : \text{int}}$$

$$\frac{\Gamma \vdash E_1 : \text{int} \quad \Gamma \vdash E_2 : \text{int}}{\Gamma \vdash E_1 - E_2 : \text{int}}$$

$$\frac{\Gamma \vdash E : \text{int}}{\Gamma \vdash \text{iszero } E : \text{bool}}$$

$$\frac{\Gamma \vdash E_1 : \text{bool} \quad \Gamma \vdash E_2 : t \quad \Gamma \vdash E_3 : t}{\text{if } E_1 \text{ then } E_2 \text{ else } E_3 : t}$$

$$\frac{\Gamma \vdash E_1 : t_1 \quad [x \mapsto t_1]\Gamma \vdash E_2 : t_2}{\Gamma \vdash \text{let } x = E_1 \text{ in } E_2 : t_2}$$

$$\frac{\Gamma \vdash E_1 : t_1 \rightarrow t_2 \quad \Gamma \vdash E_2 : t_1}{\Gamma \vdash E_1 E_2 : t_2}$$

$$\frac{[x \mapsto t_1]\Gamma \vdash E : t_2}{\Gamma \vdash \text{proc } x E : t_1 \rightarrow t_2} \quad \frac{[x \mapsto t_2, f \mapsto (t_2 \rightarrow t_1)]\Gamma \vdash E_1 : t_1 \quad [f \mapsto (t_2 \rightarrow t_1)]\Gamma \vdash E_2 : t}{\Gamma \vdash \text{letrec } f(x) = E_1 \text{ in } E_2 : t}$$

We say that a closed expression E has type t iff we can construct a proof tree for deriving $[] \vdash E : t$.

Example 1

$$\overline{[] \vdash \text{iszero } (1 + 2) : \text{bool}}$$

Example 1

$$\frac{\frac{\overline{\square \vdash 1 : \text{int}} \quad \overline{\square \vdash 2 : \text{int}}}{\square \vdash 1 + 2 : \text{int}}}{\square \vdash \text{iszero } (1 + 2) : \text{bool}}$$

Type checker proves type safety by constructing such proof trees.

Example 2

$$\frac{\frac{\overline{[x \mapsto \text{int}] \vdash x : \text{int}}}{[] \vdash \text{proc } (x) x : \text{int} \rightarrow \text{int}} \quad \overline{[] \vdash 1 : \text{int}}}{[] \vdash (\text{proc } (x) x) 1 : \text{int}}$$

Property 1 (Multiple Types)

Type assignment may not be unique.

- $\text{proc } x \ x:$

$$\frac{\overline{[x \mapsto \text{int}] \vdash x : \text{int}}}{[] \vdash \text{proc } x \ x : \text{int} \rightarrow \text{int}}$$

$$\frac{\overline{[x \mapsto \text{bool}] \vdash x : \text{bool}}}{[] \vdash \text{proc } x \ x : \text{bool} \rightarrow \text{bool}}$$

$$\frac{\overline{[x \mapsto (\text{int} \rightarrow \text{int})] \vdash x : \text{int} \rightarrow \text{int}}}{[] \vdash \text{proc } x \ x : (\text{int} \rightarrow \text{int}) \rightarrow (\text{int} \rightarrow \text{int})}$$

- $\text{proc } (f) \ (f \ 3)$ has type $(\text{int} \rightarrow t) \rightarrow t$ for any t .
- The type of $\text{proc } (f) \ \text{proc } (x) \ (f \ (f \ x))$?

Property 1 (Multiple Types)

Type assignment may not be unique.

- $\text{proc } x \ x:$

$$\frac{\overline{[x \mapsto \text{int}] \vdash x : \text{int}}}{[] \vdash \text{proc } x \ x : \text{int} \rightarrow \text{int}}$$

$$\frac{\overline{[x \mapsto \text{bool}] \vdash x : \text{bool}}}{[] \vdash \text{proc } x \ x : \text{bool} \rightarrow \text{bool}}$$

$$\frac{\overline{[x \mapsto (\text{int} \rightarrow \text{int})] \vdash x : \text{int} \rightarrow \text{int}}}{[] \vdash \text{proc } x \ x : (\text{int} \rightarrow \text{int}) \rightarrow (\text{int} \rightarrow \text{int})}$$

- $\text{proc } (f) \ (f \ 3)$ has type $(\text{int} \rightarrow t) \rightarrow t$ for any t .
- The type of $\text{proc } (f) \ \text{proc } (x) \ (f \ (f \ x))?$
 $((t \rightarrow t) \rightarrow (t \rightarrow t))$ for any t

Property 2 (Soundness)

The type system is sound.¹

- If a closed expression E is well-typed

$$\boxed{} \vdash E : t$$

for some $t \in T$, E does not have type error and produce a value.

- If E has a type error, we cannot find t such that $\boxed{} \vdash E : t$. For example:
 - ▶ $(\text{proc } (x) (\mathbf{3} \ x)) \ 1$
 - ▶ $(\text{proc } (x) (x \ \mathbf{3})) \ 4$

¹See: Benjamin C. Pierce. Types and Programming Languages. MIT Press.

Property 3 (Incompleteness)

The type system is incomplete. Even though some programs do not have type errors, they do not have types according to the type system:

- `if iszero 1 then 11 else (iszero 22))`
- `(proc (f) (f f)) (proc x x)`

Implementation of Type Checker: First Try

How to implement the type checker?

```
let rec typeof  $\Gamma$   $E$  =  
  match  $E$  with  
  |  $n \rightarrow$  int  
  |  $x \rightarrow \Gamma(x)$   
  |  $E_1 + E_2 \rightarrow$   
    let  $t_1 = \mathbf{typeof} \Gamma E_1$   
    let  $t_2 = \mathbf{typeof} \Gamma E_2$   
    if  $t_1 = \text{int}$  and  $t_2 = \text{int}$  then int  
    else raise TypeError  
  |  
  |  
  |  
  | proc  $x E \rightarrow$ 
```

Not trivial because of the following type rule:

$$\frac{[x \mapsto t_1]\Gamma \vdash E : t_2}{\Gamma \vdash \text{proc } x \ E : t_1 \rightarrow t_2}$$

Two approaches:

- **Type Annotation:** Programmers are required to supply the type of the function argument. Used in C, C++, Java, etc.
- **Type Inference:** Type checker attempts to automatically infer types. Only possible if the language is carefully designed. Used in ML, Haskell, etc.

Language with Type Annotation

Consider the language with recursive procedures:

$$\begin{array}{lcl} E & \rightarrow & n \\ & | & x \\ & | & E_1 + E_2 \\ & | & E_1 - E_2 \\ & | & \text{iszero } E \\ & | & \text{if } E_1 \text{ then } E_2 \text{ else } E_3 \\ & | & \text{let } x = E_1 \text{ in } E_2 \\ & | & \text{proc } (x : t) E \\ & | & \text{letrec } t_1 \ f(x : t_2) = E \text{ in } E \\ & | & E_1 \ E_2 \end{array}$$

In $\text{letrec } t_1 \ f(x : t_2) = E_1 \text{ in } E_2$, t_2 is the type of x and t_1 is the type of results.

Examples

- `proc (x:int) (x+1)`
- `letrec int double (x: int) =
 if iszero x then 0 else (double (x-1)) +2
in double 2`
- `proc (f: (bool -> int)) proc (n: int) (f (iszero n))`

Typing Rules

$$\frac{}{\Gamma \vdash n : \text{int}} \quad \frac{}{\Gamma \vdash x : \Gamma(x)}$$

$$\frac{\Gamma \vdash E_1 : \text{int} \quad \Gamma \vdash E_2 : \text{int}}{\Gamma \vdash E_1 + E_2 : \text{int}}$$

$$\frac{\Gamma \vdash E_1 : \text{int} \quad \Gamma \vdash E_2 : \text{int}}{\Gamma \vdash E_1 - E_2 : \text{int}}$$

$$\frac{\Gamma \vdash E : \text{int}}{\Gamma \vdash \text{iszero } E : \text{bool}}$$

$$\frac{\Gamma \vdash E_1 : \text{bool} \quad \Gamma \vdash E_2 : t \quad \Gamma \vdash E_3 : t}{\text{if } E_1 \text{ then } E_2 \text{ else } E_3 : t}$$

$$\frac{\Gamma \vdash E_1 : t_1 \quad [x \mapsto t_1] \Gamma \vdash E_2 : t_2}{\Gamma \vdash \text{let } x = E_1 \text{ in } E_2 : t_2}$$

$$\frac{\Gamma \vdash E_1 : t_1 \rightarrow t_2 \quad \Gamma \vdash E_2 : t_1}{\Gamma \vdash E_1 E_2 : t_2}$$

$$\frac{[x \mapsto t_1] \Gamma \vdash E : t_2}{\Gamma \vdash \text{proc } (x : t_1) E : t_1 \rightarrow t_2}$$

$$\frac{[x \mapsto t_2, f \mapsto (t_2 \rightarrow t_1)] \Gamma \vdash E_1 : t_1 \quad [f \mapsto (t_2 \rightarrow t_1)] \Gamma \vdash E_2 : t}{\Gamma \vdash \text{letrec } t_1 f(x : t_2) = E_1 \text{ in } E_2 : t}$$

Type Checking Algorithm

Now we can implement the type checking algorithm recursively:

```
let rec typeof  $\Gamma$   $E$  =  
  match  $E$  with  
  | proc  $(x : t_1)$   $E_1 \rightarrow$   
    let  $t_2 = \mathbf{typeof} (\{x \mapsto t_1\}\Gamma)$   $E_1$   
    in  $t_1 \rightarrow t_2$   
  | letrec  $t_1$   $f(x : t_2) = E_1$  in  $E_2$   
    let  $t' = \mathbf{typeof} (\{x \mapsto t_2, f \mapsto (t_2 \rightarrow t_1)\}\Gamma)$   $E_1$   
    in if  $t' = t_1$  then  $\mathbf{typeof} (\{f \mapsto (t_2 \rightarrow t_1)\}\Gamma)$   $E_2$   
       else raise TypeError  
  |  
  |
```

Summary

Introduction to type system.

- Basic terms: soundness, completeness, type environment.
- Typing rules for assigning types.
- Type checking algorithm with manual type annotations.

Next class: automatic type inference (i.e., type checking algorithm without type annotations).